

Autonomous Robotics

Team AR

Graded Exercise Report

February 6, 2026

Abstract

This report details the development of a unified autonomous control system for a RosBot mobile robot, designed to navigate, map, and interact with multiple distinct maze environments. The primary objective was to engineer a single, robust software architecture capable of solving variable environments without logic modification. Using a probabilistic occupancy grid for Frontier Based Exploration A* search for path planning, and computer vision for semantic object recognition, the system successfully solved 4 out of 5 test environments. The report analyzes the system's architecture, the engineering challenges encountered (such as dynamic obstacle avoidance and sensor fusion discrepancies), and the adaptive solutions implemented to achieve high-level autonomy.

1 Introduction

In the field of mobile robotics, the ability to adapt to unknown environments is the gold standard of autonomy. Hard-coding behaviors for specific layouts yields fragile systems that fail when parameters change. Consequently, this project focused on creating a **general-purpose exploration controller**.

The mission required the robot to:

1. Explore an unknown maze to locate two specific landmarks (a Blue Pillar and a Yellow Pillar).
2. Avoid static obstacles (walls) and dynamic hazards (Red Walls and Green Poison Zones).
3. Upon locating both landmarks, execute a high-speed transit (Speed Run) from the Blue Pillar to the Yellow Pillar.

Our approach utilizes a **Finite State Machine (FSM)** governing a **Probabilistic Mapping System**. This architecture allowed us to use a single controller codebase across all five mazes, adapting only minor hyperparameters (such as maximum velocity or detection thresholds) to account for differences in map scale, rather than rewriting behavioral logic.

2 System Architecture & Methodology

The controller is built upon a modular loop that integrates Sense, Plan, and Act cycles within a 32ms time step.

2.1 Probabilistic Mapping

To navigate without a pre-existing map, the robot builds a **Probabilistic Occupancy Grid**.

- **Grid Initialization:** The map consists of a 500×500 grid where each cell holds a probability value between 0.0 (Free) and 1.0 (Occupied), initialized at 0.5 (Unknown).
- **Sensor Model:** We utilized a 360-degree LiDAR sensor. For every scan, we apply **Bresenham's Line Algorithm** to trace the ray from the robot to the obstacle. Cells along the ray are decremented (probability lowered), while the endpoint is incremented (probability raised).
- **Map Inflation:** To prevent collisions during path planning, a secondary `inflated_map_grid` is generated in real-time. This layer expands occupied cells by the robot's physical radius plus a safety buffer, creating a "Configuration Space" that allows the path planner to treat the robot as a single point.

2.2 Path Planning and Navigation

We employed the **A* (A-Star) Search Algorithm** for trajectory generation.

- **Heuristic:** The algorithm uses Euclidean distance as the heuristic to find the shortest path from the robot's current coordinates to the target frontier.
- **Frontier Exploration:** The robot utilizes a "Frontier-Based Exploration" strategy. It identifies clusters of "unknown" cells adjacent to "free" space. These frontiers are scored using a weighted function:

$$Score = Distance - (0.1 \times ClusterSize) \quad (1)$$

This biases the robot to explore nearby regions first while still being attracted to large unexplored openings.

- **Visual Homing Optimization:** To mitigate aimless wandering in large corridors, we injected visual data into the planning layer. If the camera detects Blue pixels (the start goal), the scoring function applies a massive bonus to frontiers aligned with that visual bearing. This allows the robot to "see" its goal long before the LiDAR can map the path to it.

2.3 Semantic Perception (Sensor Fusion)

The robot must distinguish between structural walls and game elements (Pillars, Hazards).

- **Visual-LiDAR Fusion (Pillars):** The camera detects the color (Blue/Yellow) in HSV space. The system then correlates this 2D pixel angle with the 3D LiDAR point cloud. By filtering LiDAR points that fall within the specific angular cone of the visual detection, we accurately estimate the pillar's world coordinates.
- **Projective Geometry (Green Poison):** The Green Poison zones presented a unique challenge: they are flat textures on the floor, invisible to the LiDAR (which shoots parallel to the ground). We solved this by using **homography/projective geometry**. We mapped the 2D contour of the green blob in the camera frame to 3D world coordinates on the floor plane. This allowed us to virtually “paint” a rectangular obstacle onto the occupancy grid, effectively creating a “virtual wall” that the A* planner would naturally avoid.

3 Engineering Challenges and Solutions

Developing a single controller for varying environments introduced complex edge cases. Below are the critical problems we faced and the algorithmic solutions we engineered.

3.1 The “Red Wall” Infinite Loop

Problem: Red Walls are dangerous obstacles that require the robot to turn around. However, standard exploration logic would simply detect the wall, turn 180 degrees, and then immediately calculate a path *back* to the same unexplored area behind the red wall, causing an infinite loop.

Solution: We implemented a “**Dead End Memory**” system. When the robot visually confirms a Red Wall (via a pixel threshold check $> 5\%$), it triggers a retreat maneuver and marks a **3.0-meter radius** around that location as a “Dead End” in memory. The frontier selection algorithm is strictly forbidden from selecting targets within this radius, forcing the robot to abandon that route entirely and seek alternatives elsewhere.

3.2 Precision Docking vs. Moving Targets

Problem: Initial detections of pillars from a distance are often inaccurate. If the robot plans a path to a “noisy” coordinate seen from 5 meters away, it arrives at the wrong location.

Solution: We implemented **Dynamic Path Re-Targeting**. The robot continuously monitors the “best known location” of the target pillar. If a new detection refines the pillar's position by more than **20cm** (due to the robot getting closer and having a better view), the current path is immediately discarded, and a new A* trajectory is calculated

to the refined coordinate. This ensures the robot converges on the true location as it approaches.

3.3 The “Ghost” Poison

Problem: The Green Poison zones are rectangular carpets. Initial attempts to map them as circles resulted in the robot clipping the corners of the rectangle. Furthermore, noise in the camera projection caused the mapped shape to “jitter” or drift, confusing the planner.

Solution: We implemented **Fixed Geometry Locking**. The system calculates the bounding box of the poison zone using `cv2.minAreaRect`. Once a zone is mapped with high confidence, its position and shape are “locked” in the global map. Subsequent noisy detections are ignored if they overlap with the locked zone, ensuring a stable, safe map for the path planner.

3.4 Deadlocks and Corner Cutting

Problem: The robot would occasionally cut corners too closely, getting snagged on wall edges. Additionally, in tight spaces, it would sometimes get physically stuck.

Solution:

- **Waypoint Precision:** We tightened the `WAYPOINT_REACH_THRESHOLD` to 0.05m, forcing the robot to fully complete a movement segment before turning.
- **Two-Phase Recovery:** We designed a custom recovery state for physical collisions. Instead of spinning blindly (which fails against flat walls), the robot executes a **Force Reverse (1.0s)** to gain clearance, followed by a **180° Rotation**, successfully un-wedging itself from any geometry.

3.5 “Lost” Robot in Closed Mazes

Problem: In environments with no clear frontiers remaining (or all frontiers blocked by Red Walls), the robot would stop moving before finding the pillars.

Solution: We developed a “**Lost Mode**” fallback. If the frontier list is empty but the pillars remain unfound, the robot autonomously selects a random open point on the map, navigates there, and performs a 360-degree sensor sweep. This behavior mimics human search patterns when lost.

4 Experimental Results

The unified controller was tested across 5 environments. The table below documents the efficiency of the system. Note that “Maze 5” was the largest and most complex;

while the robot successfully mapped the majority of the environment and avoided hazards, the simulation time limit was reached before the final speed run could be completed. This is a result of the conservative safety parameters we chose to prioritize collision avoidance over raw speed.

Table 1: Performance Timings for Maze Environments

Environment	Start → Blue Pillar (s)	Blue → Yellow (s)	Outcome
Maze 1	Timeout	N/A	<i>Partial Success</i>
Maze 2	[136]	[95,71]	SUCCESS
Maze 3	[68]	[20,45]	SUCCESS
Maze 4	[128]	[53,45]	SUCCESS
Maze 5	[43]	[15,17]	SUCCESS

5 Conclusion

This project demonstrates that a single, well-architected controller can handle significant environmental variance. By combining **probabilistic SLAM**, **robust state-machine logic**, and **projective semantic mapping**, we created a system that is not just a “maze solver,” but a capable autonomous explorer. The minor parameter tuning required for different mazes reflects real-world robotics, where physical dynamics (speed, friction) must always be tuned to the specific operating domain, while the core intelligence remains universal.